University course pack

Teaching programming and algorithms at university

A 14-week, 3-credit module in data structures and algorithms for second-year computer science students. Prerequisites: introductory programming and discrete mathematics. The four Darsi lectures anchor weeks 1, 2, 4 and 6 and frame the course's assessment design.

Learning outcomes

1. Use O, Θ and Ω precisely and solve common recurrences.
2. Design recursive algorithms and prove them correct by induction.
3. Trace programs accurately using a viable notional machine.
4. Implement and compare standard data structures empirically and analytically.
5. Test and debug code systematically against edge cases.

14-week syllabus

Week	Topic	In class and out
1	Notional machines and tracing	Diagnostic tracing quiz
2	Asymptotic analysis	Lab 1: time two sorts
3	Arrays, lists and amortised analysis	Problem set 1
4	Recursion and induction	Lab 2: be the stack
5	Divide and conquer and the master theorem	Problem set 2
6	Testing, debugging and autograding	Assignment 1 with hidden tests
7	Stacks, queues and hashing	Lab 3: hash tables
8	Trees and binary search trees	Midterm exam
9	Balanced trees and heaps	Problem set 3
10	Graphs and traversal	Assignment 2
11	Shortest paths	Lab 4: Dijkstra
12	Dynamic programming	Problem set 4
13	Greedy algorithms and intractability	Code-reading and oral checks
14	Review	Final exam

University course pack

Lecture slide outline

Lecture 1: Asymptotic analysis

- Correct code that never finishes
- Timing two sorts
- O, Θ and Ω formally
- Recurrences and the master theorem
- Activity: time it, then prove it

Lecture 2: Recursion and induction

- A factorial that crashes
- Frames on the call stack
- Termination measures
- Induction mirrors the code
- Activity: be the stack

Lecture 3: Notional machines

- The a = b surprise
- Du Boulay's notional machine
- Sorva's misconception catalogue
- Trace, explain, write
- Activity: write diagnostic questions

Lecture 4: Assessment at scale

- Visible and hidden tests
- Mutation testing the suite
- Feedback that names the case
- Integrity when AI can code
- Activity: break the submission

University course pack

Problem set

1. Prove that $2n^2 + 3n = O(n^2)$ by giving c and n_0, and show that it is not O(n).

2. Solve $T(n) = 3T(n/2) + n$ and $T(n) = 2T(n/2) + n^2$ with the master theorem.

3. Write a recursive function for the sum of a list and prove it correct by induction on the length.

4. How many calls does naive fib(6) make, and what does memoisation reduce this to?

5. Write a multiple-choice tracing question with three distractors, each mapped to a named misconception.

6. A binary search passes the visible tests. List four hidden tests you would add.

Solutions

1. Take $c = 3$, $n_0 = 3$: $2n^2 + 3n \le 3n^2$ when $n \ge 3$. If $2n^2 + 3n \le cn$ for all large n, then $2n + 3 \le c$, impossible as n grows.

2. First: $n^{(\log_2 3)} \approx n^{1.585}$ dominates n, so $\Theta(n^{(\log_2 3)})$. Second: n^2 dominates $n^{(\log_2 2)} = n$ and the regularity condition holds, so $\Theta(n^2)$.

3. sum([]) = 0; sum(x:xs) = x + sum(xs). Base: the empty list sums to 0. Step: assuming sum(xs) is correct, x + sum(xs) is the sum of x:xs.

4. C(n) = C(n-1) + C(n-2) + 1 with C(0) = C(1) = 1 gives C(6) = 25. With memoisation each value from 0 to 6 is computed once: 7 computations, $\Theta(n)$.

5. Example: a = 3; b = a; a = a + 1; print(a, b). Correct: 4 3. Distractors: 4 4 (assignment as link), 3 3 (statements not executed in order), 3 4 (confuses which variable changed).

6. An empty list; a target smaller than every element; a target equal to the last element; a list of 10^6 items with a time limit to detect a linear scan.

Lab handout: Lab 1: Time it, then prove it

Goal

Connect measured running time to asymptotic bounds for two sorting algorithms.

Steps

1. Implement insertion sort and merge sort.
2. Time both on random inputs from 10 to 10^4 items, then merge sort alone up to 10^6. Time the small sizes many times over (for example with timeit).
3. Plot time against n on log-log axes and estimate the slopes.
4. Find the input size where the curves cross.
5. Derive each bound and explain any gap with your measurements.

Hand in

Code, a plot, the crossover size and a two-page report.

University course pack

Assessment rubric

Criterion	Excellent	Good	Developing	Not yet
Correctness	Passes all, with own edge-case tests	Passes most hidden tests	Passes visible tests	Fails visible tests
Analysis	Derived and related to measurements	Bound derived	Bound stated	No bound
Proof	Rigorous	Complete with minor gaps	Structure only	Missing
Explanation	Explains and modifies live	States purpose	Describes lines	Cannot explain

Reading list

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
Core text.

du Boulay, B. (1986). Some difficulties of learning to program. Journal of Educational Computing Research, 2(1), 57–73.
The notional machine.

Sorva, J. (2013). Notional machines and introductory programming education. ACM Transactions on Computing Education, 13(2), Article 8.
Misconception catalogue.

Lister, R., et al. (2004). A multi-national study of reading and tracing skills in novice programmers. ACM SIGCSE Bulletin, 36(4), 119–150.
Tracing before writing.

Porter, L., Bailey Lee, C., & Simon, B. (2013). Halving fail rates using peer instruction: A study of four computer science courses. Proceedings of SIGCSE '13, 177–182.
Peer instruction evidence.

Ala-Mutka, K. M. (2005). A survey of automated assessment approaches for programming assignments. Computer Science Education, 15(2), 83–102.
Autograding.

Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. Proceedings of SIGMOD '03, 76–85.
The MOSS algorithm.

Prather, J., et al. (2023). The robots are here: Navigating the generative AI revolution in computing education. Proceedings of ITiCSE-WGR '23, 108–159.
AI and programming assessment.